Yenepoya university Art , Commerce & Science
Project – Low Level Design
on
SmartCity ID — Secure Identity Framework

Student Name: Sayed Abdullah Suadaan

Industry mentor : Mr shashank

Guide by : Ms Vijayalaxmi

Table of Contents

Introduction. 3
- Scope of the document. 3
- Intended Audience 3
- System overview. 3

System Design. 4
- Application Design 4
- Process Flow. 4
- Information Flow. 5
- Components Design 5
- Key Design Considerations 6
- API Catalogue. 6

Data Design. 6
- Data Model 6
- Data Access Mechanism 7
- Data Retention Policies 7
- Data Migration 7

Interfaces 8

State and Session Management 9

Caching 9

Non-Functional Requirements 10
- Security Aspects 10
- Performance Aspects 10

References 11

1. Introduction

This Low Level Design (LLD) document provides the complete technical design specification for the SmartCity ID — Secure Identity Framework. It defines module internals, function-level logic, data structures, interface elements, validation rules, and security mechanisms in sufficient detail to guide direct implementation. The SmartCity ID system is a browser-based Role-Based Access Control (RBAC) application contained in a single self-contained HTML file. It requires no server, no internet connection, and no external dependencies.

1.1. Scope of the document

This document covers the complete implementation design of the SmartCity ID system including the following areas:

- Authentication flow: registration, login credential verification, session cleanup on logout
- Role-Based Access Control: role routing at the APP level and function-level guards at BACKEND level
- Citizen Dashboard: personal identity display, city status auto-refresh every 5 seconds, emergency banner
- Admin Control Room: tabbed overview, live audit log rendering every 4 seconds, user table management
- Backend Simulation: password hashing algorithm, traffic level logic, time condition logic, system state
- Data Design: user schema, log schema, status schema, data access patterns, retention rules
- Interface Design: all four UI views with element IDs, input types, validation rules, and refresh rates
- Security Design: dual-layer role enforcement, input validation, session isolation, audit trail
- Error Handling: all failure scenarios with detection layer, user feedback, and system response

1.2. Intended Audience

This document is prepared for the following readers: Software Developers responsible for implementing the system, Quality Assurance Engineers designing test cases based on the specified behaviour, Academic Evaluators and Faculty assessing the technical quality of the project design, and Industry Mentors from IBM and NCS reviewing the document for professional standards compliance.

1.3. System overview

The SmartCity ID system provides a two-role digital identity portal for a simulated smart city environment. Citizens register and log in to view their personal identity record and city status metrics in read-only mode. Administrators log in to access system status, live audit logs, and registered user management. All logic runs in JavaScript within the browser. The BACKEND module acts as a simulated server, encapsulating the in-memory database and all business logic inside an IIFE (Immediately Invoked Function Expression) so that the private data store is inaccessible from outside the module. The APP module acts as the application controller, managing routing, event listeners, and DOM updates.

2. System Design

The SmartCity ID follows a Single Page Application (SPA) architecture. All four views — Login, Register, Citizen Dashboard, and Admin Control Room — reside within a single HTML file and are shown or hidden by toggling the CSS class 'active' on each page div. The application separates concerns across three layers: the HTML structure (views), the APP module (controller), and the BACKEND module (model/business logic).

2.1. Application Design

The four views and their access rules are defined as follows:

Page / View	Page ID	Accessible By	Purpose
Login Page	login-page	All users (unauthenticated)	Entry point — username and password authentication form
Register Page	register-page	New users (unauthenticated)	Citizen self-registration with field validation and district selection
Citizen Dashboard	citizen-page	Citizens only	Read-only personal identity record and city status monitoring
Admin Control Room	admin-page	Admin only	System overview, audit logs tab, and user management tab

2.2. Process Flow

The end-to-end process flow for user interactions with the SmartCity ID system is described below:

- User opens application — Login Page is shown as default active view
  - First-time user: clicks 'Register' link → completes registration form → redirected to Login Page after 1.5s
  - Returning user: enters credentials on Login Page → clicks Authenticate button
- APP.handleLogin() reads username and password from DOM fields, calls BACKEND.login(username, password)
  - If username not found → WARNING logged, error alert shown, form shakes
  - If password hash does not match → WARNING logged, error alert shown, form shakes
  - If credentials valid → safe user object returned (no password field)
- APP checks returned user.role:
  - role === 'citizen' → initCitizenDashboard() called → citizen-page shown
  - role === 'admin' → initAdminDashboard() called → admin-page shown
- Citizen Dashboard: personal info populated once on login; city status polled every 5 seconds via setInterval
- Admin Control Room: status + logs polled every 4 seconds; sensor data refreshed every 6 seconds
- User clicks Logout → clearIntervals() called → currentUser set to null → login-page shown

2.3. Information Flow

Data flows between system layers in a strictly controlled, one-directional pattern. The BACKEND DB object is private and never directly accessible from outside the IIFE.

Flow Direction	From	To	Data Passed
User → APP	Browser form inputs	APP event handlers	Username, password, registration field values
APP → BACKEND	APP.handleLogin()	BACKEND.login()	Plain text username and password string
BACKEND → APP	BACKEND.login()	APP.handleLogin()	{ success, user } — user object with password stripped
APP → DOM	APP.initCitizenDashboard()	HTML element textContent	Personal info fields, city status values
BACKEND → APP	BACKEND.getSystemStatus()	APP update functions	traffic, timeCondition, AI status, power, security, timestamp
BACKEND → APP	BACKEND.getLogs()	APP.renderLogs()	Array of up to 50 log entry objects (copy, not reference)
BACKEND → APP	BACKEND.getUsers()	APP.renderUsers()	Array of user objects — password field stripped from each

2.4. Components Design

The system comprises the following key components, each with a defined responsibility boundary:

Component	Type	Responsibility
BACKEND Module	JavaScript IIFE	Encapsulates DB object. Exposes: login, register, getSystemStatus, getLogs, getUsers, addLog, triggerEmergency, changeSecurityLevel, toggleAIBrain, refreshSensors
APP Module	JavaScript IIFE	Controls page routing (showPage), event listeners (login/register/logout forms), dashboard initialisation, DOM updates (setIfExists), interval management
DB Object	Private JS Object	In-memory store: users[] array, logs[] array, emergencyActive flag, systemPower, securityLevel, aiBrainStatus, sensorData object
hashPassword()	Private function	Deterministic rolling hash converting plain text to HC$ prefixed 12-char hex string — called by register() and login()
getTrafficLevel()	Private function	Returns LOW/MEDIUM/HIGH string based on new Date().getHours() — called on every getSystemStatus() invocation
getTimeCondition()	Private function	Returns MORNING_SHIFT/AFTERNOON_SHIFT/EVENING_SHIFT/NIGHT_SHIFT based on current hour
showPage(id)	APP function	Removes 'active' class from all .page elements; adds 'active' and 'page-enter' to target page for CSS transition
renderLogs()	APP function	Maps BACKEND.getLogs() array to color-coded HTML template strings; assigns to log-container innerHTML in one write
renderUsers()	APP function	Maps BACKEND.getUsers() array to HTML table row strings; assigns to users-tbody innerHTML in one write
clearIntervals()	APP	Calls clearInterval() on sensorInterval, clockInterval, statusInterval;

Component	Type	Responsibility
	function	prevents background polling after logout

2.5. Key Design Considerations

The following architectural decisions were made to ensure the system is secure, portable, mobile-compatible, and maintainable:

- Single File Architecture: The entire application — HTML, CSS, and JavaScript — is embedded in one self-contained file. This eliminates dependency on relative file paths and web servers, which commonly cause loading failures when opening HTML projects on mobile devices or sharing via WhatsApp and email.
- IIFE Encapsulation Pattern: Both BACKEND and APP use the IIFE pattern. The DB object and all internal functions are scoped inside the IIFE closure, making them inaccessible from the browser console. This simulates server-side data privacy — citizens cannot read or modify DB through browser developer tools.
- No Persistent Storage by Design: localStorage and sessionStorage are intentionally not used. All session state is held in module-level closure variables that reset when the tab is closed. This prevents XSS-based session theft and ensures no personal data is left on shared or public devices.
- Dual-Layer Role Enforcement: Role is verified at two independent checkpoints — the APP router after login and inside each admin-only BACKEND function. Even if a citizen called an admin BACKEND function directly from the console, it would return ACCESS DENIED.
- Offline-First Design: All fonts, styles, and scripts are embedded inline — zero external network calls. The application functions identically with or without internet. This was a direct response to mobile device failures caused by Google Fonts CDN calls.
- Mobile-Responsive Layout: CSS Grid with breakpoints at 600px and 380px. All inputs set to font-size: 16px to prevent iOS Safari auto-zoom on focus. All touch targets set to minimum 44px height.

2.6. API Catalogue

The BACKEND module exposes the following public API. All internal helpers (hashPassword, generateSensors, getTrafficLevel, getTimeCondition) are private:

Function Signature	Returns	Access Level
login(username, password)	{ success: bool, user?: object, message?: string }	Public — called by APP
register(u, p, fullName, email, dist, pin)	{ success: bool, message: string }	Public — called by APP
getSystemStatus()	Status object: traffic, time, power, security, AI, emergency, timestamp	Public — called by APP
getLogs()	Copy of DB.logs array (max 50 entries)	Public — called by APP
getUsers()	DB.users array with password field stripped	Public — called by APP
addLog(type, message, level)	void — prepends entry to DB.logs	Public — called internally
triggerEmergency(adminUser)	{ success, emergencyActive, message }	Admin only — role checked

Function Signature	Returns	Access Level
changeSecurityLevel(adminUser, level)	{ success, securityLevel }	Admin only — role checked
toggleAIBrain(adminUser)	{ success, aiBrainStatus }	Admin only — role checked
refreshSensors()	Updated sensorData object	Public — called by admin interval

3. Data Design

All application data is stored in-memory inside the BACKEND module's private DB object. No external database, file system access, browser localStorage, or sessionStorage is used. All data is session-scoped and is lost when the browser tab is closed or the page is refreshed. The DB contains three data collections: a users array, a logs array, and system state flags.

3.1. Data Model

User Object Schema — each entry in DB.users follows this structure:

Field	Type	Required	Constraints	Example
id	Integer	Yes	Auto-incremented from 1	2
username	String	Yes	Min 3 chars; unique in DB.users	john_doe
password	String	Yes	HC$ hash — plain text never stored	HC$00A1B2C3D4E5
role	String	Yes	Enum: 'admin' or 'citizen' only	citizen
fullName	String	Yes	Non-empty display name	John Doe
email	String	No	Auto-generated if blank	john.doe@smartcity.gov
district	String	No	Kerala or Karnataka district name	Kasaragod, Kerala
pincode	String	No	Exactly 6 digits if provided	671121
joined	String	Yes	ISO 8601 date: YYYY-MM-DD	2025-04-15

Log Entry Schema — each entry in DB.logs follows this structure:

Field	Type	Description	Example
id	Integer	Sequential counter, incremented per addLog() call	12
timestamp	String	toLocaleTimeString() at moment of event	9:45:22 AM
date	String	toLocaleDateString() at moment of event	4/15/2025
type	String	Category: LOGIN, LOGIN_FAIL, REGISTER, LOGOUT, EMERGENCY, SECURITY, AI_BRAIN, SYSTEM	LOGIN_FAIL
message	String	Human-readable event description	Wrong password for: john_doe
level	String	INFO (blue badge), WARNING (amber), CRITICAL (red)	WARNING

3.2. Data Access Mechanism

All data access is mediated through the BACKEND public API. The following access rules are enforced by design and cannot be bypassed from outside the IIFE:

- getUsers() strips the password field using ES6 destructuring ({ password: _, ...safeUser }) before returning any user object. Passwords never leave the BACKEND module.
- login() calls hashPassword(input) and compares the result to the stored hash. The stored hash is never returned. On success, a password-stripped user copy is returned.
- getLogs() returns a copy of DB.logs using the spread operator ([...DB.logs]) — external mutation of the internal array is prevented.
- getSystemStatus() assembles and returns a fresh object on every call — no internal DB references are exposed in the returned object.
- Admin write functions (triggerEmergency, changeSecurityLevel, toggleAIBrain) each check adminUser.role === 'admin' independently before modifying any DB field and return ACCESS DENIED if the check fails.

3.3. Data Retention Policies

Since only in-memory storage is used, all data is session-scoped and ephemeral:

- User data: Exists in DB.users for the current browser session only. Registrations are lost on page refresh.
- Log data: DB.logs is capped at 50 entries. Each addLog() call checks if DB.logs.length > 50 and removes the oldest entry with pop() before appending the new entry with unshift().
- System flags: emergencyActive, securityLevel ('ALPHA'), and aiBrainStatus ('ONLINE') reset to default values on every page load as part of the DB object initialisation.

3.4. Data Migration

This system does not involve data migration as there is no persistent storage layer. If this system were extended to a production deployment, the following migration path would apply:

- User data would be migrated to a relational database (PostgreSQL) with bcrypt or Argon2 password hashing replacing the current rolling hash.
- Audit logs would be migrated to a time-series store (Elasticsearch or InfluxDB) with long-term retention and search capabilities.
- System state flags would be moved to Redis for distributed state management across multiple server instances.

4. Interfaces

The SmartCity ID system exposes four user interface views, all sharing the same futuristic dark theme: deep navy background, cyan accent color, monospaced display typography, and animated UI elements. Each view is documented below with its key UI elements, DOM IDs, data sources, and refresh behaviour.

View	DOM ID	Role	Key Elements
Login Page	login-page	All	Username input, password input with eye-toggle, Authenticate button, alert box, Register link, demo credentials
Register Page	register-page	New users	Full name, username, district dropdown (14 Kerala + 30 Karnataka), pincode, email, password, confirm password, Register button, alert box
Citizen Dashboard	citizen-page	Citizens	Welcome banner, personal identity table (username/role/email/district/pincode/joined), 4 city status cards, emergency banner, live clock, logout
Admin Control Room	admin-page	Admin	4-stat header strip, 3 tabs: Overview (emergency state, AI orb, user count), Logs (color-coded audit log, 4s refresh), Users (full user table)

Login Page — detailed element specification:

Element	Type	DOM ID	Behaviour
Username	Text input	login-username	Required — non-empty check in APP before BACKEND call; validated client-side
Password	Password input	login-password	Required — masked text; eye-button toggles type between password and text
Authenticate btn	Submit button	login-form	Fires handleLogin(); red shake animation on failure; green alert + 1s redirect on success
Alert box	Div	login-alert	Shows error (red) or success (green) message; auto-hides after 4 seconds via setTimeout
Register link	Anchor	—	Onclick: hides login-page, shows register-page (CSS class toggle)

Citizen Dashboard — element IDs, data sources, and refresh rates:

DOM ID	Data Source	Refresh	Notes
c-welcome	currentUser.fullName	On login	Welcome, {fullName}

DOM ID	Data Source	Refresh	Notes
c-username	currentUser.username	On login	Read-only identity field
c-role	currentUser.role.toUpperCase()	On login	Displays CITIZEN
c-fullname	currentUser.fullName	On login	Read-only identity field
c-email	currentUser.email	On login	Read-only — auto-generated if not set
c-district	currentUser.district	On login	Shows 'District, State' format
c-pincode	currentUser.pincode	On login	Shows — if not provided
c-joined	currentUser.joined	On login	ISO date YYYY-MM-DD
c-ai-status	getSystemStatus().aiBrainStatus	5s	ONLINE or STANDBY
c-power	getSystemStatus().systemPower	5s	Shown as percentage e.g. 94%
c-security	getSystemStatus().securityLevel	5s	ALPHA / BETA / GAMMA / DELTA / OMEGA
c-traffic	getSystemStatus().trafficLevel	5s	LOW/MEDIUM/HIGH with CSS color class
c-emergency-banner	getSystemStatus().emergencyActive	5s	display:flex when true, display:none when false
c-clock	new Date().toLocaleTimeString()	1s	Live clock in dashboard header

Admin Control Room — tab elements, data sources, and refresh rates:

Tab / Element ID	Data Source	Refresh	Notes
Overview — a-emergency-state	getSystemStatus().emergencyActive	4s	STANDBY (green) or ACTIVE (pulsing red)
Overview — a-ai-status	getSystemStatus().aiBrainStatus	4s	ONLINE or STANDBY
Overview — a-user-count	BACKEND.getUsers().length	4s	Total registered users
Overview — a-timestamp	getSystemStatus().timestamp	4s	Full locale date+time string
Logs — log-container	BACKEND.getLogs() array	4s	Color-coded divs: blue=INFO, amber=WARNING, red=CRITICAL
Users — users-tbody	BACKEND.getUsers() array	On init	Columns: ID, Username, Full Name, Role badge, District, Pincode, Joined
Header — a-admin-name	currentUser.fullName	On login	Admin display name in header

Tab / Element ID	Data Source	Refresh	Notes
Header — a-clock	new Date().toLocaleTimeString()	1s	Live clock

5. State and Session Management

Session state is managed entirely within APP module closure variables. No browser storage APIs (localStorage, sessionStorage, cookies) are used anywhere in the system. This ensures all session data is automatically cleared when the tab is closed or refreshed, preventing session hijacking from shared devices.

Variable	Type	Lifecycle	Description
currentUser	Object | null	Set on login; null on logout or page refresh	Safe user object from BACKEND.login() — password field stripped
sensorInterval	Number | null	Set on dashboard init; cleared by clearIntervals()	ID of setInterval polling getSystemStatus() every 5s (citizen) or sensor refresh every 6s (admin)
clockInterval	Number | null	Set on dashboard init; cleared by clearIntervals()	ID of setInterval updating clock display every 1 second
statusInterval	Number | null	Set on admin init; cleared by clearIntervals()	ID of setInterval polling admin status and rendering logs every 4 seconds

On logout, handleLogout() calls clearIntervals() which invokes clearInterval() on all three timer IDs, then sets currentUser to null and navigates to login-page. This guarantees no background polling continues after the user session ends, preventing both memory leaks and stale data display.

6. Caching

The SmartCity ID system does not require application-level caching because all data already resides in-memory within the BACKEND module's private DB object. All read operations — getLogs(), getUsers(), getSystemStatus() — are direct in-memory reads executing in microseconds with no I/O overhead. The system status values (traffic level, time condition, timestamp) are recomputed fresh on every call to getSystemStatus() using the system clock, ensuring accuracy without cache invalidation complexity.

The audit log array (DB.logs) functions as a self-managing in-memory ring buffer. Entries are prepended at the front using unshift() (newest first) and the oldest is dropped with pop() when the 50-entry cap is exceeded. This means the displayed log set is always current and requires no flush or sync operation.

Data Type	Storage	Read Strategy	Write / Update
User records	DB.users array	Array.find() — O(n)	BACKEND.register() appends; never mutated after creation
Audit logs	DB.logs array	Spread copy — O(n)	BACKEND.addLog() prepends; pop() enforces 50-entry cap
System status	Computed on demand	Clock read — O(1)	Admin functions mutate DB flags; recomputed on next getSystemStatus()
Session state	APP closure vars	Direct read — O(1)	Set on login; null on logout; auto-cleared on page refresh
Sensor data	DB.sensorData	Object read — O(1)	refreshSensors() regenerates random values every 6s

7. Non-Functional Requirements

Non-functional requirements define the quality, performance, usability, and compatibility standards that the SmartCity ID system must satisfy across all supported platforms and usage contexts.

7.1. Security Aspects

Security is the primary non-functional concern for an identity and access management system. The SmartCity ID design incorporates the following security controls:

Security Control	Implementation	Threat Mitigated
Password Hashing	Rolling polynomial hash (HC$ prefix). Plain text never stored in DB.users.password.	Credential exposure if DB object is inspected
Hash-Only Comparison	login() hashes the input and compares to stored hash. The hash is never returned to APP layer.	Timing attacks; plain-text comparison vulnerabilities
Password Stripped	getUsers() and login() use destructuring to strip password field before returning any user object.	Accidental password leakage through API responses
Role Enforcement — APP	After login(), APP router checks user.role and sends to correct page. No URL routing means no bypass path.	Unauthorised access to admin-page via navigation
Role Enforcement — API	Each admin BACKEND function checks adminUser.role === 'admin' first. Returns ACCESS DENIED if check fails.	Privilege escalation via browser console function calls
No External Requests	Zero CDN, API, or font calls — all assets embedded inline in single HTML file.	Man-in-the-middle attacks; CDN supply chain injection
Session in Closure	currentUser held in IIFE closure — not in localStorage or any globally accessible browser API.	XSS-based session token theft from browser storage
Audit Trail	addLog() records timestamp, type, and severity for every auth event (login, fail, register, logout).	Lack of accountability and forensic audit evidence
Input Validation	APP validates before BACKEND call; BACKEND re-validates independently. Dual-layer prevents bypass.	Malformed data; injection-equivalent attacks on DB
No Auto-Persist	Session cleared on tab close or refresh — no remember-me or persistent token functionality.	Session hijacking from shared or public devices

7.2. Performance Aspects

Since all data is in-memory and the application has no network dependencies, performance is inherently strong. The following targets are defined and met by the implementation:

Requirement	Target	How Achieved
Login response time	< 100ms	In-memory hash compare and array find — zero network or I/O latency

Requirement	Target	How Achieved
Registration response	< 100ms	Array push with hash computation — entirely in RAM
Dashboard load time	< 200ms	DOM innerHTML assignments only — no virtual DOM or framework overhead
Status poll latency	< 50ms	setInterval reads in-memory DB object — negligible CPU cost
Log render (50 entries)	< 50ms	Single Array.map to template string + one innerHTML write
User table render	< 30ms	O(n) map over users array — typically fewer than 20 users
Application file size	< 80 KB	Single self-contained HTML file — no external libraries or frameworks
Animation smoothness	60 fps	CSS animations use transform/opacity only — GPU-composited layers, no layout reflow
Mobile load time	Instant	file:// protocol — zero HTTP round-trips; no server required
Interval cleanup	On logout	clearIntervals() calls clearInterval() on all 3 IDs — no background CPU drain after logout

8. References

No.	Reference	Source
1	SmartCity ID — HLD Document (IBM Format)	HLD_AND_LLD_FORMAT_-_IBM.docx provided by faculty
2	MDN Web Docs — Web Audio API	developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
3	MDN Web Docs — Document Object Model	developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
4	MDN Web Docs — IIFE Pattern	developer.mozilla.org/en-US/docs/Glossary/IIFE
5	MDN Web Docs — setInterval / clearInterval	developer.mozilla.org/en-US/docs/Web/API/setInterval
6	OWASP — Authentication Cheat Sheet	cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
7	OWASP — Input Validation Cheat Sheet	cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
8	NIST SP 800-63B — Digital Identity Guidelines	pages.nist.gov/800-63-3/sp800-63b.html
9	NIST — Role-Based Access Control	csrc.nist.gov/projects/role-based-access-control
10	Apple Human Interface Guidelines — Touch Targets	developer.apple.com/design/human-interface-guidelines
11	Kerala Districts — Government of Kerala Official Portal	kerala.gov.in
12	Karnataka Districts — Govt of Karnataka Official Portal	karnataka.gov.in

— End of Low Level Design Document —

SmartCity ID | LLD v1.0 | Academic Year 2025–2026